# Text File Handling in Java

19 August 2026 09:43

Text file handling means creating, opening, reading, writing, appending, modifying and deleting files that contain data in the form of characters or human-readable text.

Java provides several classes for text - file handling through packages such as: java.io and in modern Java: ***java.nio.file***

For beginners, the java.io classes such as FileReader, FileWriter, BufferedReader and BufferedWriter are particularly useful

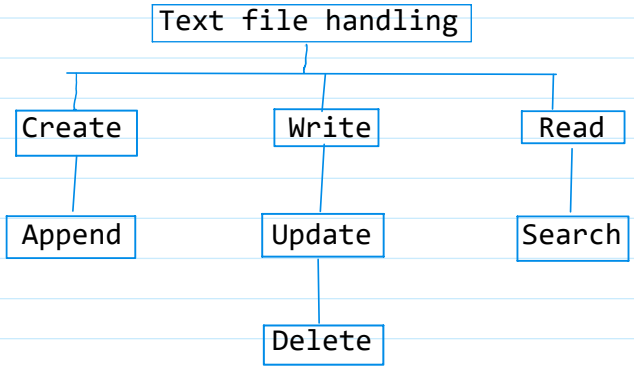
**What is a Text file?**
A text file stores information as characters that can normally be viewed using a text editor.

Examples:
- Student.txt
- Employees.txt
- Marks.txt
- Data.csv

For example, student.txt might contain:

101, Ajay, 87
102, Anchit, 90
103, Faisal, 78

**Text file handling Operations**
A Java program can perform several operations on text files:




**Common operations are:**
**i. Create a file.**
**ii. Write data.**
**iii. Read the data.**
**iv. Append data.**
**v. Search data.**
**vi. Update data.**
**vii. Delete data.**

viii. **Rename a file.**
ix. **Check file properties.**

**Important Java Classes**

**File:** Represents a file or directory.
It can be used to:
i. Check whether a file exist.
ii. Create a file.
iii. Delete a file.
iv. Get file name.
v. Get file size.
vi. Check whether it is a directory.

**FileReader:** Used to read character data from a text file.

**FileWriter:** Used to write character data to a text file.

**BufferedReader:** Reads text efficiently, particularly line by line.

**BufferedWriter:** Writes text efficiently.

**PrintWriter:** Provides convenient methods such as:
print()
println()
printf()
For writing text

**Writing to what text file using FileWwriter**

```
package FileHandling;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;
public class WtiteDataInFile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileWriter fw = new FileWriter(path, true)){
            fw.write("101, Rahul, 85\n");
            fw.write("102, Anchit, 95\n");
            fw.write("103, Aaditya, 55\n");
            fw.write("104, Raman, 65\n");
            fw.write("105, Aman, 75\n");
            System.out.println("Data written successfully.");
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
```

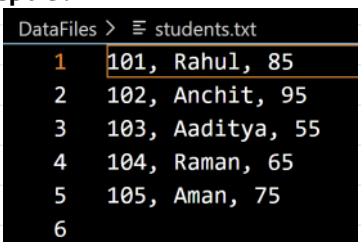
'append' option is activated.

This is called try-with-resource.

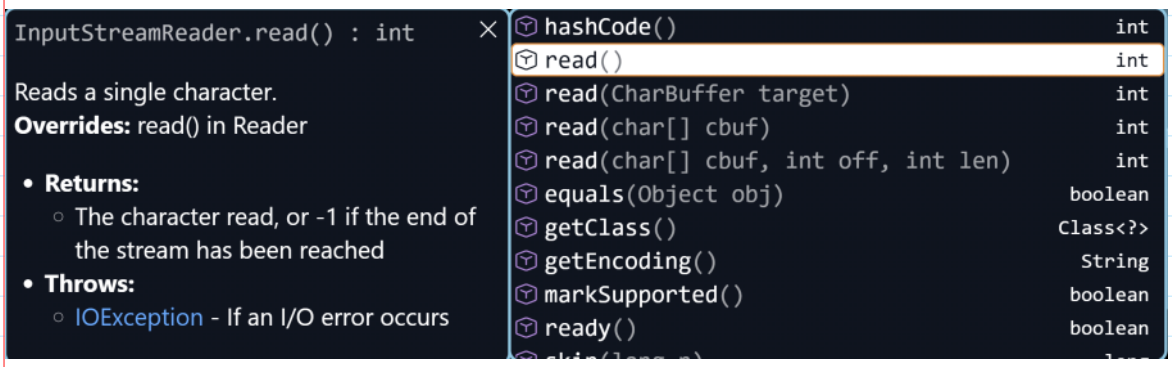
↑ Java automatically closes fw when the try block finishes.
It is generally preferable to manual calling close().

```
            sc.close();
        }
    }
}
```

**Output:**




## Reading a text file using FileReader

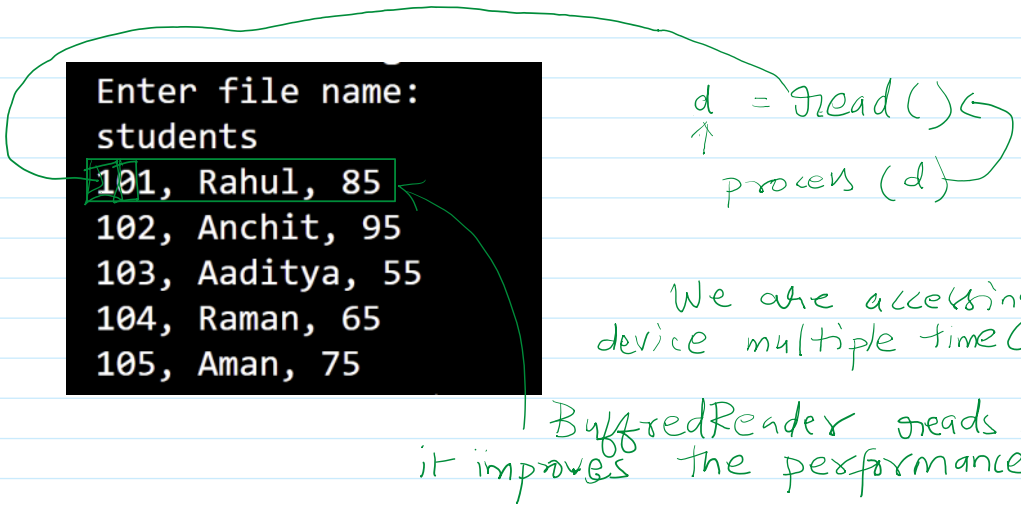



```
package FileHandling;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;
public class ReadDataFromTextfile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileReader fr = new FileReader(path)){
            int d;
            while((d = fr.read()) != -1){
                System.out.print((char)d);
            }
        }
        catch(FileNotFoundException fnfe){
            fnfe.printStackTrace();
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
```

```
            sc.close();
        }
    }
}
```

**Output:**

```
Enter file name:
students
101, Rahul, 85
102, Anchit, 95
103, Aaditya, 55
104, Raman, 65
105, Aman, 75
```




```
BufferedReader.readLine() : String
```

Reads a line of text. A line is considered to be terminated by any one of a line feed ('\n'), a carriage return ('\r'), a carriage return followed immediately by a line feed, or by reaching the end-of-file (EOF).

- **Returns:**
  - A String containing the contents of the line, not including any line-termination characters, or null if the end of the stream has been reached without reading any characters
- **Throws:**
  - IOException - If an I/O error occurs
- **See Also:**
  - java.nio.file.Files.readAllLines

```
package FileHandling;
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;
public class ReadingFileUsingBufferedReader {
```

```
public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileReader fr = new FileReader(path);
    BufferedReader br = new BufferedReader(fr)){
            String line;
            while((line = br.readLine()) != null){
                System.out.println(line);
            }
        }
        catch(FileNotFoundException fnfe){
            fnfe.printStackTrace();
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
            sc.close();
        }
    }
}
```

Output:

```
Enter file name:
students
101, Rahul, 85
102, Anchit, 95
103, Aaditya, 55
104, Raman, 65
105, Aman, 75
```

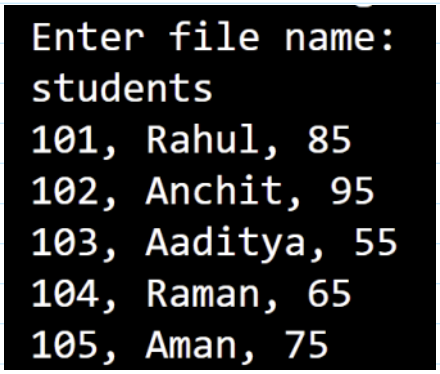
**Reading a file into a String**

We can use Files.readString() in modern Java.

```
package FileHandling;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Scanner;
public class ReadingFileInAString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try{
            String data = Files.readString(Path.of(path));
```

```
            System.out.println(data);
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
            sc.close();
        }
    }
}
```

Output:



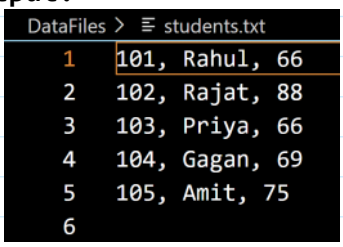

**Writing Using BufferedWriter**

**Code:**

```
package FileHandling;
import java.io.BufferedWriter;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;
public class BufferedWriterDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileWriter fw = new FileWriter(path, true);
    BufferedWriter bw = new BufferedWriter(fw)){
            do{
                System.out.println("Enter roll number: ");
                int rn = sc.nextInt();
                sc.nextLine();
                System.out.println("Enter name: ");
                String name = sc.nextLine();
                System.out.println("Enter marks: ");
                int marks = sc.nextInt();
                sc.nextLine();
```

```
                    bw.write(rn+", "+name+", "+marks);
                    bw.newLine();
                    System.out.println("Press Y/y to store more records: ");
                    String ch = sc.next();
                    if(ch.equalsIgnoreCase("y")){
                        continue;
                    }
                    else{
                        break;
                    }
                }while(true);
            }
            catch(FileNotFoundException fnfe){
                fnfe.printStackTrace();
            }
            catch(IOException e){
                e.printStackTrace();
            }
            finally{
                sc.close();
            }
        }
    }
```

**Output:**




**Using PrintWriter:** Printer Writer is a convenient when writing formatted text.

Code:

```
package FileHandling;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;
public class PrintWriterDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter file name: ");
        String filename = sc.nextLine();
        String path = "DataFiles/"+filename+".txt";
        try(FileWriter fw = new FileWriter(path, true);
    PrintWriter pw = new PrintWriter(fw)){
            do{
                System.out.println("Enter roll number: ");
```

```
                int rn = sc.nextInt();
                sc.nextLine();
                System.out.println("Enter name: ");
                String name = sc.nextLine();
                System.out.println("Enter marks: ");
                int marks = sc.nextInt();
                sc.nextLine();
                // pw.println(rn+", "+name+", "+marks);
                pw.printf("%d, %s, %d%n", rn, name, marks);
                System.out.println("Press Y/y to store more records: ");
                String ch = sc.next();
                if(ch.equalsIgnoreCase("y")){
                    continue;
                }
                else{
                    break;
                }
            }while(true);
        }
        catch(FileNotFoundException fnfe){
            fnfe.printStackTrace();
        }
        catch(IOException e){
            e.printStackTrace();
        }
        finally{
            sc.close();
        }
    }
}
```

Output:

DataFiles > students.txt

```
1   101, Rahul, 66
2   102, Rajat, 88
3   103, Priya, 66
4   104, Gagan, 69
5   105, Amit, 75
6   106, Jiya, 99
7   107, Anchit, 100
```

WAP to perform a search operation by taking roll_number as input. If found display the record, otherwise print record not found.

**Reading a File into a String**

We can use Files.readString() in modern Java.

The Files class is present in java.nio.file package.

java.lang.Object
java.nio.file.Files

public final class Files extends Object
This class consists exclusively of static methods that operate on files, directories, or other types of files.
In most cases, the methods defined here will delegate to the associated file system provider to perform the file operations.
Since:
1.7

From <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/nio/file/Files.html>

Use the above link to read more about Files class methods.

For example:

```
package FileHandling;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
public class ReadFileInString {
    public static void main(String[] args) {

        try {
            String data =
Files.readString(Path.of("DataFiles/students.txt"));
            System.out.println(data);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
101, Rahul, 66
102, Rajat, 88
103, Priya, 66
104, Gagan, 69
105, Amit, 75
106, Jiya, 99
107, Anchit, 100
```

This is convenient when the entire file is reasonably small.

```
package FileHandling;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
public class ReadAllLines {
    public static void main(String[] args) {
        try {
            List<String> lines =
Files.readAllLines(Path.of("DataFiles/students.txt"));
```

```
            for(String line : lines){
                System.out.println(line);
            }
        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
}
```

```
Output:
   101, Rahul, 66
   102, Rajat, 88
   103, Priya, 66
   104, Gagan, 69
   105, Amit, 75
   106, Jiya, 99
   107, Anchit, 100
```

**Updating a record in a Text File**

Text files generally do not provide convenient in - place modification of a line when the replacement changes its length.

A common Approach is:

```
   Original File
      |
      V
   Read line by line
      |
      V
   Modify the required record
      |
      V
   Write everything in temporary file
      |
      V
   Delete the original file
      |
      V
   Rename the temporary file with original file
```

```
package FileHandling;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;
public class ModifyRecord {
```

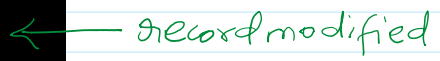
```
    public static void main(String[] args) {
        int roll, marks;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter roll number  to modify: ");
        roll = sc.nextInt();
        System.out.println("Enter new marks: ");
        marks = sc.nextInt();
        File original = new File("DataFiles/students.txt");
        File tempFile = new File("DataFiles/tempStudent.txt");
        try (BufferedReader br = new BufferedReader( new
FileReader(original));
                BufferedWriter bw = new BufferedWriter(new
FileWriter(tempFile))) {
            String line;
            while((line = br.readLine())!= null){
                String[] parts = line.split(", ");
                if(Integer.parseInt(parts[0]) == roll){
                    line = parts[0]+", " + parts[1]+", " + marks;
                }
                bw.write(line);
                bw.newLine();
            }
        }
        catch(IOException ioe){
            ioe.printStackTrace();
        }
        if(original.delete()){
            if(tempFile.renameTo(original)){
                System.out.println("Record updated.");
            }
            else{
                System.out.println("Could not rename temprory file.");
            }
        }
        else{
            System.out.println("Could not delete the original file.");
        }
    }
}
```

Output:

```
Enter roll number  to modify:
104
Enter new marks:
75
```

DataFiles > students.txt

```
1   101, Rahul, 66
2   102, Rajat, 88
3   103, Priya, 66
4   104, Gagan, 75
5   105, Amit, 75
6   106, Jiya, 99
7   107, Anchit, 100
```



Homework:
Use the above concept to delete a particular record from the file student.txt